

Orientation, set-up, and your first pull request

Environment, tooling and the GitHub workflow

Dinu-Ștefan Rusu

Week 2

Lab objectives



Install a working toolchain

Flutter and Dart on your laptop, with flutter doctor reporting no blocking issues.

TIME

2 hours



Run an app on a device

An emulator or simulator that boots, with the default Flutter app running on it.

SUBMIT

Push to `admd-labs`, branch `lab-1`, before Lab 2

The cumulative lab app

1. One repository per student, named `admd-labs`.
2. One branch per lab. One pull request per lab, merged into `main`.
3. Create the branch and install tools before starting.

Seven labs build one app that grows from empty to networked and authenticated.

Lab 1 sets up the machine. Lab 7 is the test. Labs 2 to 6 add features.

Grading

WORK	DESCRIPTION	POINTS
Colloquium	End of semester, before the exam session	20
Midterm	Middle of the semester	20
Lab assignments	Labs 1 to 6, cumulative app	30
Practical test	Lab 7, 90 minutes, individual	30
Pass mark		50 of 100

Hint

Your grader checks the repository history, not just the final state. Commit early and often, with clear messages.

Installation links

```
# Flutter SDK:  
  https://docs.flutter.dev/install  
# Add flutter/bin to your PATH  
  
flutter --version  
flutter doctor -v  
  
# Android Studio or Xcode  
# Pick one IDE: VS Code or IntelliJ
```

BEFORE STARTING

Install Flutter, an IDE, and either Android Studio or Xcode.

TARGET

The current stable Flutter channel.

IMPORTANT

Type URLs exactly. They are complete on one line.

Environment setup

1. Install Flutter from docs.flutter.dev/install.
2. Follow Install manually for your OS.
3. Add flutter/bin to your PATH.
4. Install Android Studio or Xcode (macOS).
5. Install an editor: VS Code or IntelliJ.
6. Run `flutter doctor`, fix every blocking issue.

DONE WHEN

- ☐ `flutter --version` works in a new terminal.
- ☐ `flutter doctor` shows no red warnings for Flutter or Android.
- ☐ You can explain one warning you fixed.

Running the default app

1. Boot an emulator or simulator.
2. Run `flutter devices` to confirm it appears.
3. Create the project:
`flutter create lab1_app`.
4. Run it: `flutter run`.
5. Change the app title string in `lib/main.dart`.
6. Press `r` to hot-reload and see it update.
7. Take a screenshot.

DONE WHEN

- ☐ The counter app runs and the button increments it.
- ☐ Your edited title appears after hot reload.
- ☐ A screenshot is in `/screenshots`.

flutter doctor errors

cmdline-tools missing

Android Studio > SDK Manager > SDK Tools > Android SDK Command-line Tools. Install, then run `flutter doctor` again.

License unknown

Run `flutter doctor --android-licenses` and answer `y` to all prompts.

Unable to locate SDK

Open Android Studio once. The SDK downloads on first launch. On Mac, accept the Xcode license first.

Emulators and devices

Emulator never boots

Enable hardware acceleration (VT-x / AMD-V) in BIOS. On Apple silicon Macs, pick arm64 images only.

No devices listed

Run `flutter devices`. Start the emulator with `flutter emulators --launch <id>`. Physical Android phones work with USB debugging on.

Xcode but no simulator

Open Xcode once and accept the license, then run `open -a Simulator`. If it still fails, run `xcode-select --install`.

The pull request workflow

1. Create a repository on GitHub.
2. Clone it: `git clone <url>`.
3. Create a branch: `git checkout -b lab-N`.
4. Work on your code, commit often with clear messages.
5. Push the branch: `git push -u origin lab-N`.
6. Open a pull request on GitHub to merge into `main`.
7. Merge it, so the next lab starts from `main`.

First pull request

```
git add .  
git commit -m "Lab 1: app runs"  
git push -u origin lab-1
```

IN THE PR DESCRIPTION

Add your screenshot and write: *"App runs and hot reload works."*

On GitHub, open a PR from lab-1 to main. Paste your screenshot in the description.

Submission

Push to branch `lab-1` with a screenshot and one commit